EE5203 L03 Practice Worksheet

Functions, arrays, strings, and multiple C files

Basri Erdogan

Expected time: 30-45 minutes

Policy: Attempt every question before consulting the worked solutions. Answer on paper first; the debugger is for verifying, not discovering.

Part A - Function contracts

1. For each declaration, state the return type, the function name, and each parameter's type, then read the contract aloud in one sentence.

```
int  clamp(int value, int low, int high);
bool make_status(char destination[], int capacity, uint16_t mean);
void acquire_samples(uint16_t out[], int count);
```

2. Write declarations (no bodies) for:

   a. a function that receives a temperature in tenths of a degree (`int`) and returns `true` when it is inside a safe range;
   b. a function that receives an array of `uint16_t` readings and its count, and returns the smallest reading;
   c. a function that receives nothing and returns nothing, and restarts a watchdog timer.

3. Trace the call:

```
int scale(int x, int gain) { return x * gain; }

int a = 6;
int b = scale(a + 2, 3);
```

   a. What values do the parameters receive?
   b. What is `b`?
   c. What is `a` after the call, and why?

Part B - Declare, define, call, and the build

4. Match each build message to its stage and the most likely fix:

Message	Stage (compiler/linker)	Likely fix
`conflicting types for 'clamp'`		
`undefined reference to 'count_above'`		
`implicit declaration of function 'make_status'`		

5. A project has `sensor_math.h`, `sensor_math.c`, and `main.c`. State which file(s) must contain each of these lines, and why:

```
uint16_t average_samples(uint16_t values[], int count);   /* (i)   */
#include "sensor_math.h"                                  /* (ii)  */
uint16_t average_samples(uint16_t values[], int count)    /* (iii) */
{ ... }
```

6. `#define SAMPLE_COUNT 8` appears in a header. A student claims this uses two bytes of RAM. Correct the claim and explain what actually happens to `uint16_t samples[SAMPLE_COUNT];` before compilation.

Part C - Pass by value

7. This function is intended to zero small signals, but it has no effect at the call site. Explain why, then fix it **using a return value** (pointers arrive next week).

```c
void apply_deadband(int signal, int width)
{
    if (signal > -width && signal < width) {
        signal = 0;
    }
}
```

8. Predict what the debugger shows for `x`, `y`, and `result`, immediately after the call returns:

```c
int add_offset(int x) { x = x + 10; return x; }

int x = 5;
int y = 2;
int result = add_offset(y);
```

Part D - Arrays, counts, and widths

9.
```c
   uint16_t values[] = {18, 24, 17, 31, 22, 19};
```

   Write `uint16_t maximum_sample(uint16_t values[], int count)`. State the starting value of your `maximum` variable, the first loop index, and your `count == 0` policy.

10. `average_samples` sums eight clamped samples (each at most `4095U`).

    a. What is the largest possible sum for 8 samples? Does it fit in `uint16_t`?
    b. From which sample count can a `uint16_t` sum overflow?
    c. Why does the lab still require `uint32_t`?

11. A student calls `average_samples(bounded_samples, 9)` on an eight-element array. The build is clean and a value is returned. Explain what the function actually read, and why this defect produces no build message.

12. Trace the pipeline by hand for the four-element input `{100, 5000, 2600, 300}` with limit `4095`, threshold `2500`, average over 4:

Result	Value
bounded array	
sum	
average	
count above	
status (`<1500` LOW, `>3000` HIGH, else OK)	

Part E - Strings

13. Draw the contents of `char text[8]` after `"LOW"` is stored: one row per index 0-7, showing the character or byte where defined. Which indices hold specified values?

14. Explain the difference between these two comparisons and state which one the lab's oral check accepts:

```c
if (command == "START")             { ... }
if (strcmp(command, "START") == 0) { ... }
```

15. `make_status(text, 4, 3500U)` must select `"HIGH"`.

a. How many bytes does `"HIGH"` need?
b. What should the function write, and what should it return?
c. Why is checking `capacity` against the **selected** word better than checking against the longest possible word?

16. In the Memory view, `status_text` shows `4F 4B 00 ....` Decode the bytes and state what each one proves about the program.

Part F - Debugger evidence

17. Your claim: "`average_samples` used all eight clamped samples." Describe the breakpoint placement, the two variables to watch, and the values you expect at the first and last pause that would prove the claim.

Exit self-assessment

Mark one:

☐ Ready for the L03 lab without additional review
☐ Need to review declarations vs. definitions and the build stages
☐ Need to review pass by value and return values
☐ Need to review array parameters and counts
☐ Need to review C strings and the terminator